Sign once. Verify at every boot.

Replace the on-device CMAC with OEM signature verification at startup. ECDSA now, ML-DSA as the target, one key chain for flashing and boot.

~5 ms	~13 ms	0
ECDSA hash + verify, 2 MB image	ML-DSA hash + verify, 2 MB image	secret keys stored on the ECU
MEASURED	MEASURED	PUBLIC ANCHOR ONLY

RECOMMENDATION

Verify the OEM signature at boot. Drop the on-device CMAC.

ONE SIGNATURE

One signature over the image, verified at flashing and again at every boot.

NO SECRET ON THE ECU

The private key stays in the OEM backend HSM. The ECU holds only the public key anchor.

PQC PATH BUILT IN

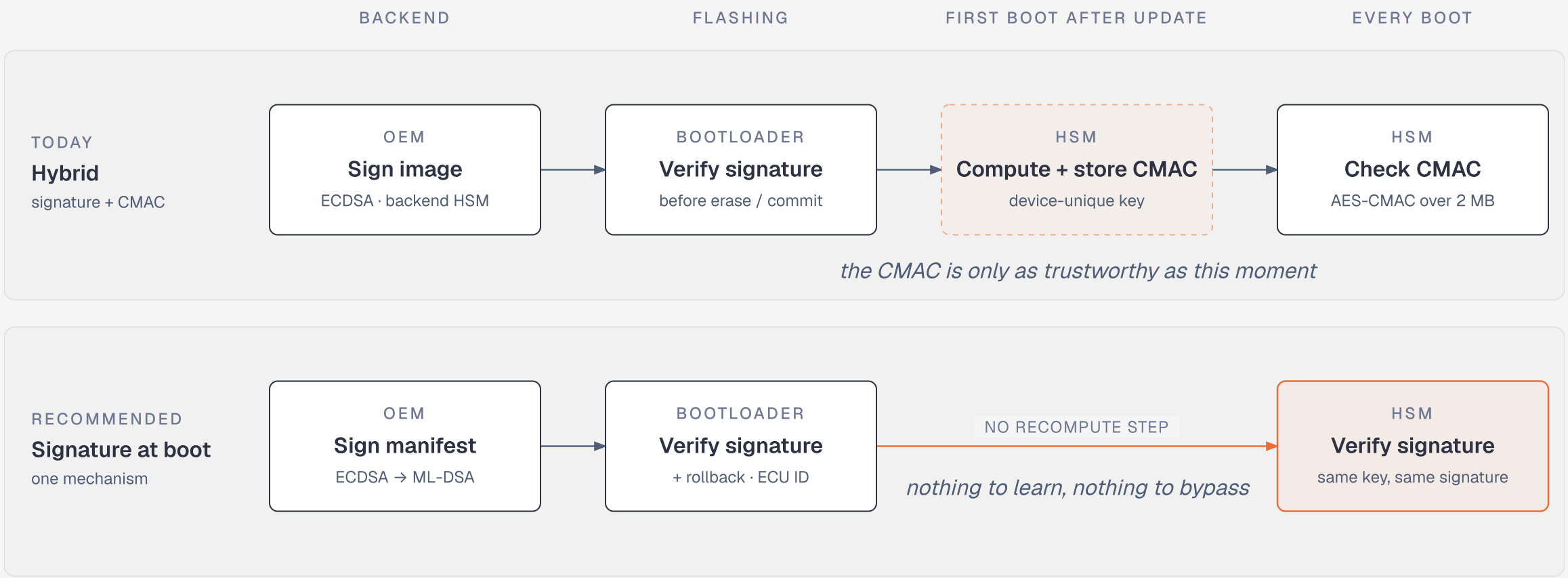
ECDSA today, ML-DSA as the target. Anchors for both are provisioned from day one.

KEEP THE HYBRID ONLY WHERE

- the measured boot-time delta does not fit the startup budget, even with staged verification
- the ECU is SHE-class with no ECC/SHA acceleration, for example TC3xx

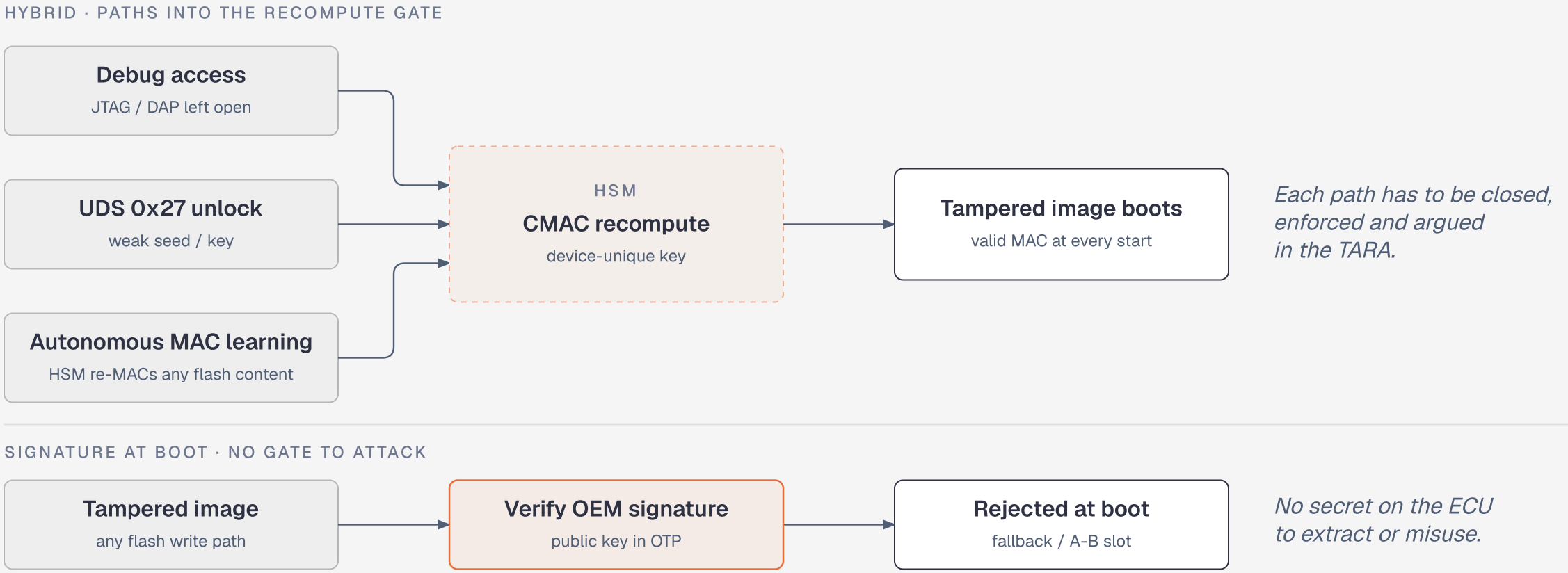
“The CMAC is only as trustworthy as the moment it was computed.”

Two boot models. One step makes the difference.

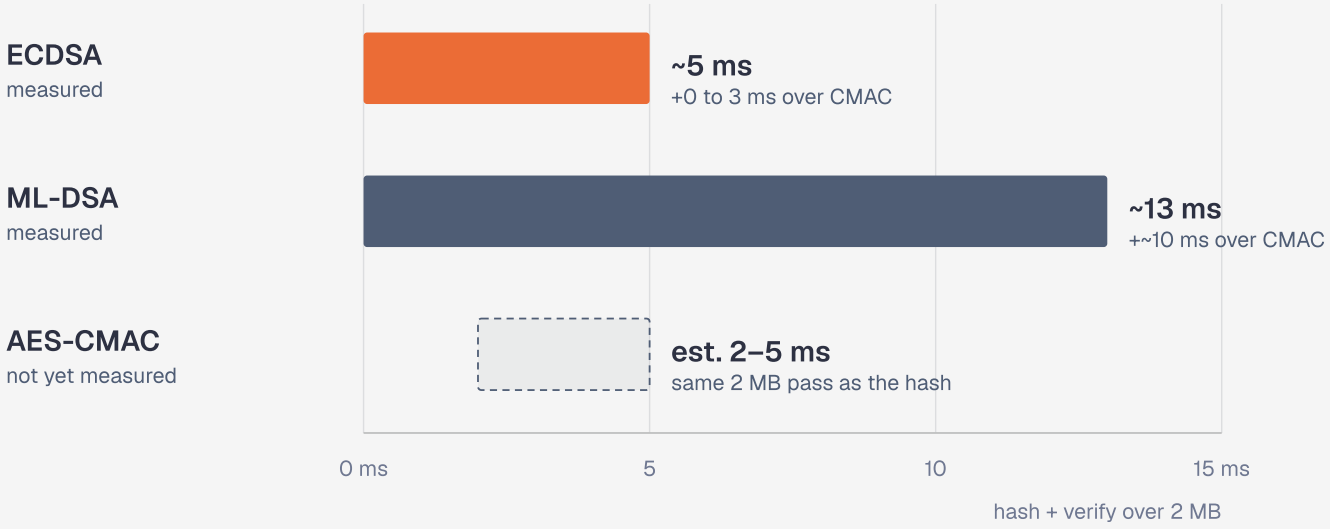


LEGEND — Recommended boot check · Residual-risk step · Step

The recompute gate is the hybrid's largest residual risk.



On TC4xx the boot-time argument for CMAC is weak.



HASH DOMINATES

On a 2 MB image most of the time is the pass over the data. A CMAC must also read all 2 MB, so it lands in the same range as the hash share.

THE REAL GAP

Expected cost over CMAC: **+0 to 3 ms** for ECDSA, **about +10 ms** for ML-DSA.

SCALING

If SHA and AES-CMAC throughput are similar, the delta is roughly the fixed verify cost and does not grow with image size.

OPEN

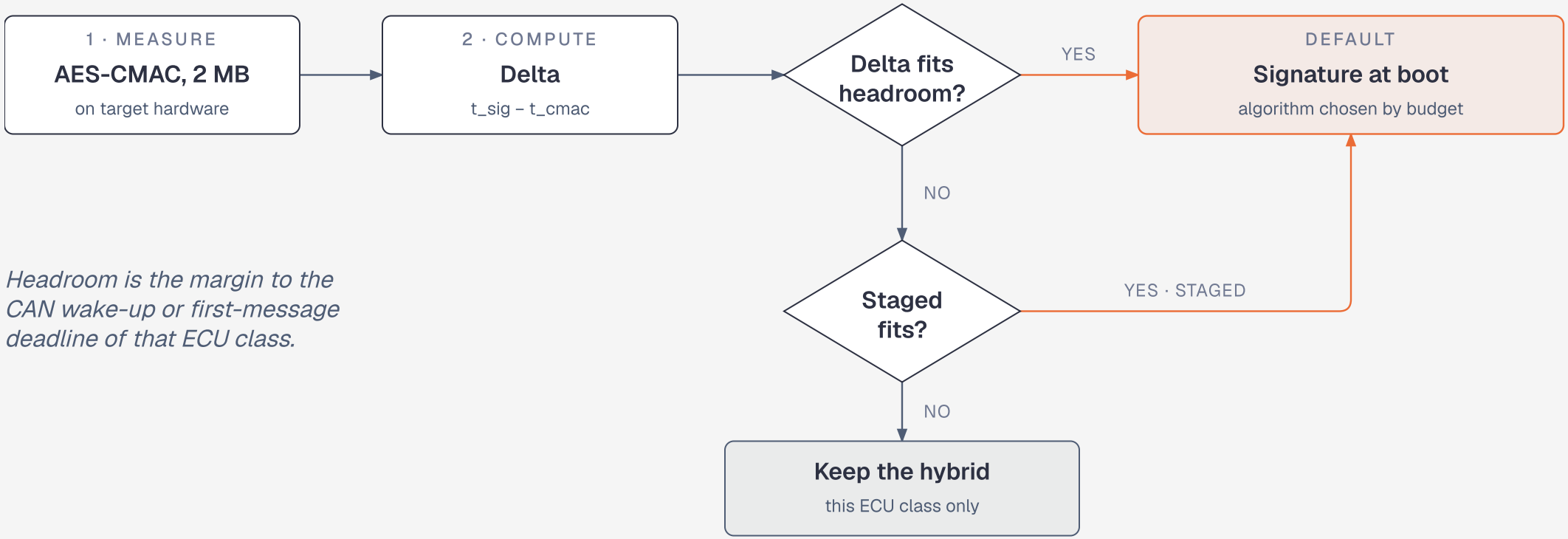
AES-CMAC over 2 MB is not measured yet. Its bar is an estimate derived from the expected gaps.

COMPARISON

Signature at boot proves more and stores nothing secret.

CRITERION	ON-DEVICE CMAC (HYBRID)	SIGNATURE AT BOOT
What is proven at boot	Image unchanged since the MAC was computed	Image released by the OEM key holder
Secret stored on the ECU	Device-unique MAC key	None, public key only
Recompute gate after update	**Yes · main residual risk**	No
MAC learning or bypass via debug / UDS 0x27	Must be controlled	Not applicable
Mechanisms to argue in TARA / R156	Two: signature + CMAC	One
Boot time on TC4xx	Fastest	+0 to 3 ms ECDSA · +~10 ms ML-DSA
PQC at boot	Not needed (AES)	Feasible, ML-DSA measured at 13 ms

Decision rule per ECU class.



Headroom is the margin to the CAN wake-up or first-message deadline of that ECU class.

METHOD · ALGORITHM CHOICE

Pick the algorithm by budget. Keep the verifier crypto-agile.

BUDGET ALLOWS ~13 MS

ML-DSA now

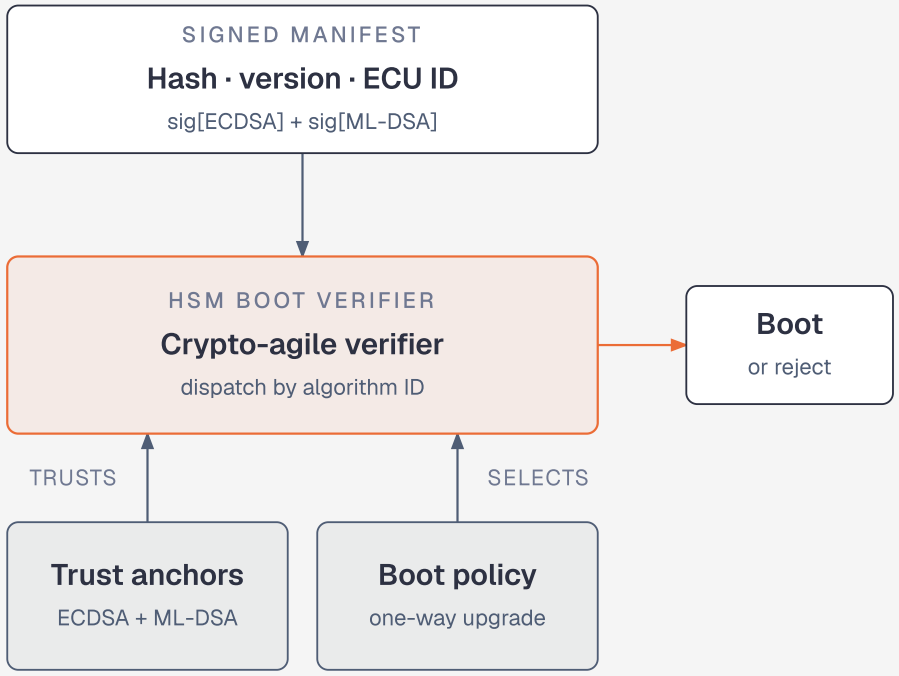
Or an ECDSA + ML-DSA dual signature during the transition.

BUDGET IS TIGHT

ECDSA now

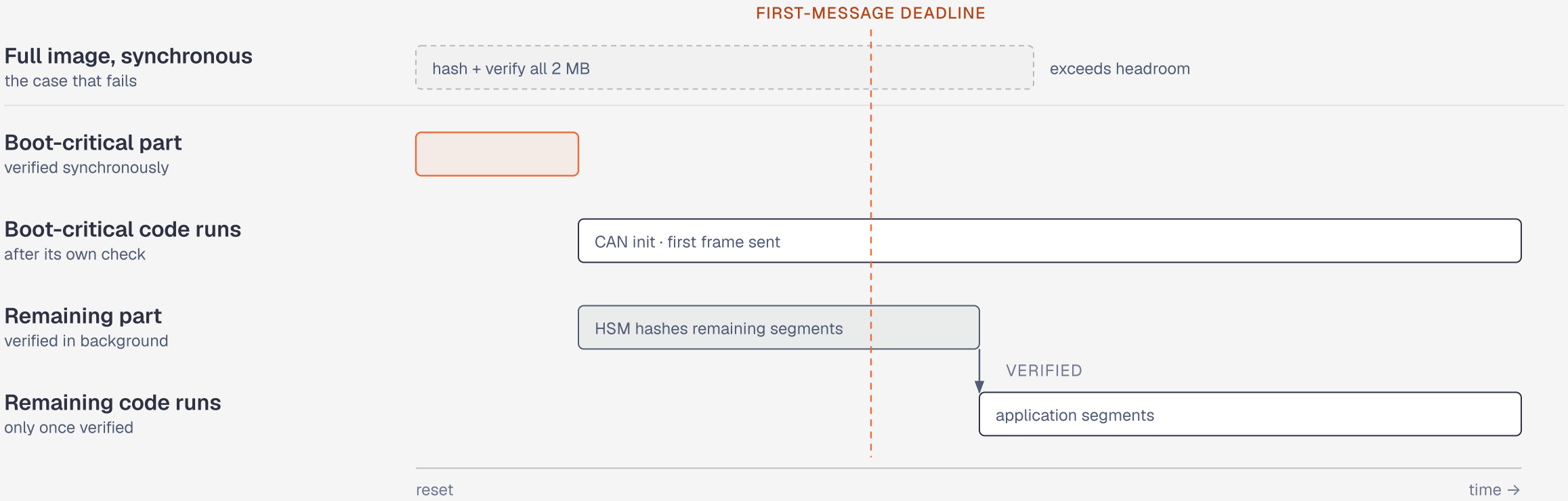
Ship a crypto-agile verifier and provision PQC anchors from day one.

A dual-signed manifest carries both signatures. The boot policy decides whether one or both must pass.



OPTION · STAGED VERIFICATION

Try staged verification before falling back to the hybrid.



MANIFEST

One signature covers a manifest that lists a hash per segment.

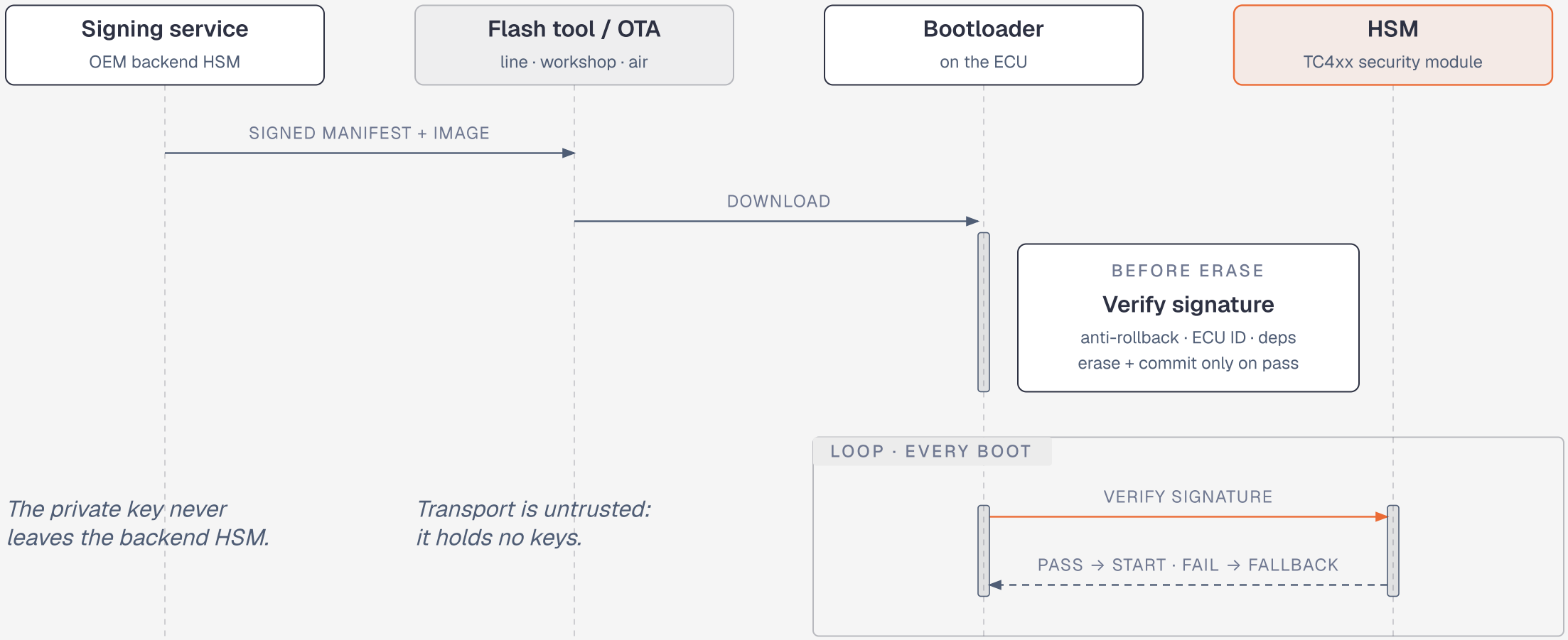
EXECUTION GATE

Later segments cannot run until their hash checks out.

FAILURE PATH

A background failure resets to the fallback image.

One signature, checked twice.



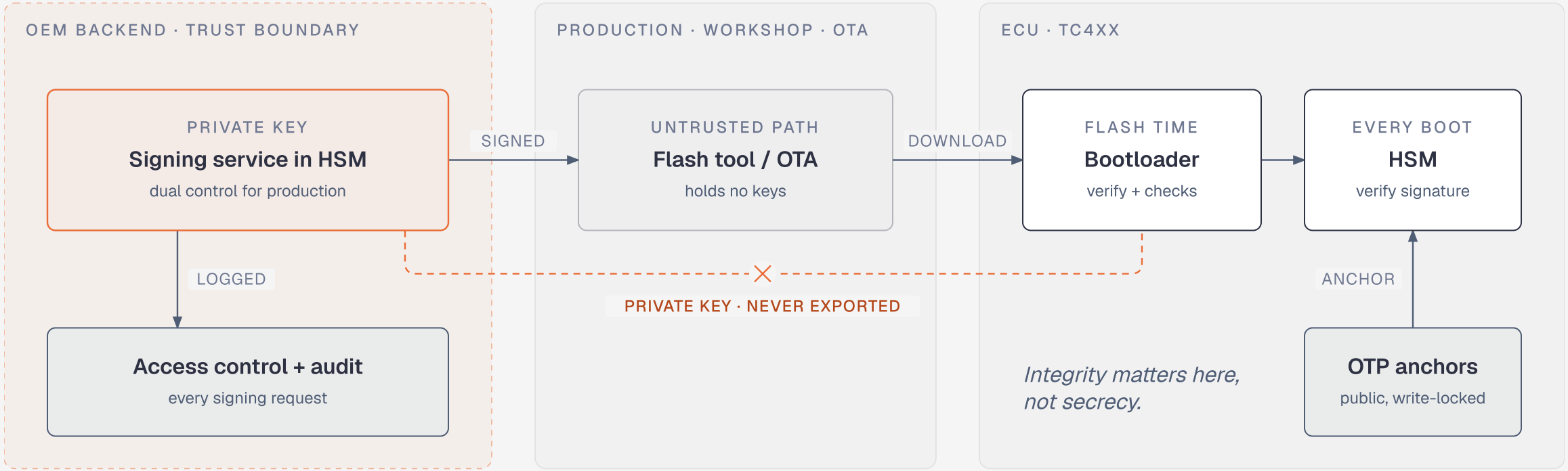
Keep the flash-time check. It guards what boot cannot.

```
// signed manifest
image_hash      hash of the full image
version         anti-rollback value
target_ecu_id   ECU / variant
dependencies    required versions
segment_hashes  optional, for staged boot
signature       ECDSA | ML-DSA | both
```

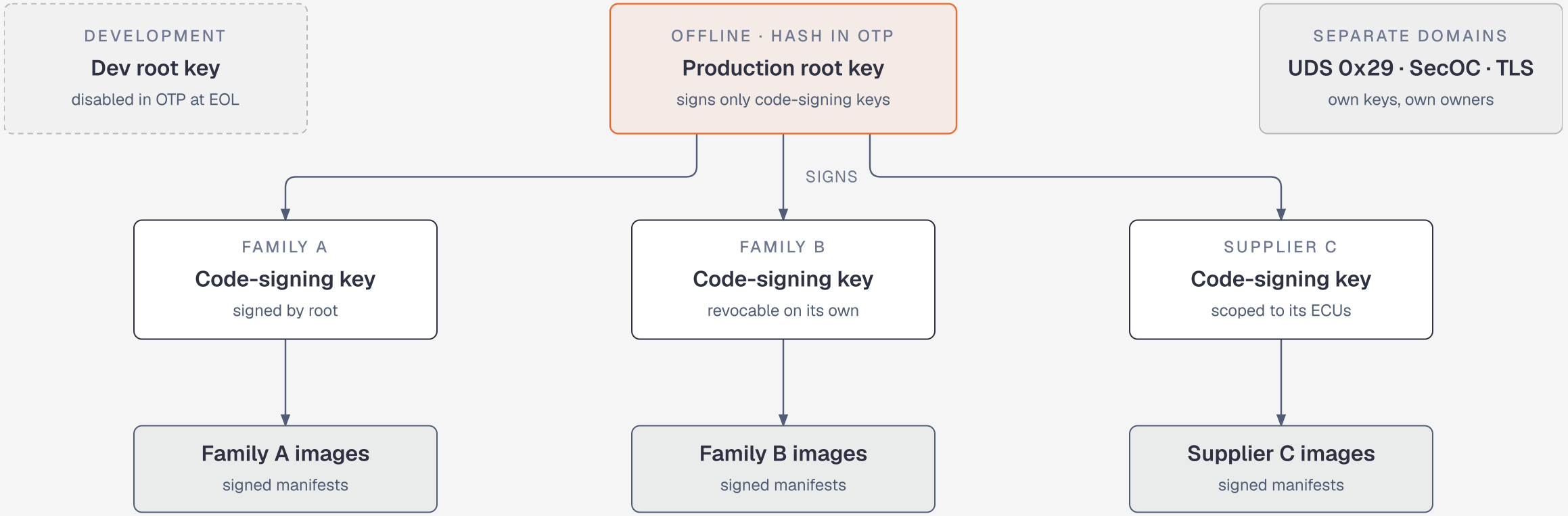
CHECK	AT FLASHING	EVERY BOOT
Signature over the manifest	Yes	Yes
Image matches the signed hash	Yes	Yes
Anti-rollback version counter	Yes	Not needed
Target ECU ID / variant	Yes	Not needed
Manifest dependencies	Yes	Not needed
Reject before the working image is erased	Yes	n/a

Rejecting a bad image before erase means no bricking: the A/B slot or fallback stays intact.

Private key in the backend. Public anchor on the ECU.



A key hierarchy that survives a 15-year vehicle life.



Reserve the OTP layout on day one.

ANCHOR 0	**ECDSA root key hash**	active at SOP
ANCHOR 1	**ML-DSA root key hash**	provisioned now, enforced later
ANCHOR 2-3	Reserved root slots	rotation and recovery
REVOCATION	**Key revocation counter**	counts up only
ANTI-ROLLBACK	**Image version counter**	checked at flashing
BOOT POLICY	**ECDSA → dual → ML-DSA**	one-way upgrade
LIFECYCLE	**Dev keys off · write-protect**	set at end of line

IMMUTABLE, NOT SECRET

Anchors are public. What matters is that nobody can change them after end of line.

ONE DIRECTION ONLY

Every field that changes in the field moves one way: counters up, policy toward PQC.

BUDGET FOR CHANGE

Expect at least one key rotation in a 15+ year life, plus the ECDSA to ML-DSA switch.

Proposed layout. Slot count and sizes to be fixed against the HSM's OTP map.

ML-DSA becomes a policy switch, not a field OTP change.

NOW · SOP

ECDSA enforced

Both anchors in OTP. Crypto-agile verifier shipped.

TRANSITION

Dual signature

Backend signs each manifest with ECDSA and ML-DSA. ML-DSA checked where the budget allows.

POLICY SWITCH

ML-DSA enforced

One-way policy update. No OTP change on vehicles in the field.

THROUGH LIFE

Rotate and retire

Rotate code-signing keys via reserved slots. Disable dev keys at end of line.

SCOPE · EXCEPTIONS

Where the hybrid stays, and what it then needs.

ECU CLASS A

SHE-class HSMs

For example TC3xx. No ECC or SHA acceleration, so signature verification at every boot is too slow.

ECU CLASS B

TC4xx outside the budget

Classes where the measured delta misses the startup budget, even with staged verification.

IF THE HYBRID STAYS, CONTROL THE GATE

- Recompute the CMAC only right after a successful signature check
- No autonomous MAC learning by the HSM
- Debug locked in production
- UDS 0x27 cannot trigger a recompute alone
- Argue both mechanisms in TARA and R156

ROLLOUT

Checklist before rollout.

MEASURE

- [ ] AES-CMAC benchmark on TC4xx (2 MB) completed and delta computed
- [ ] Startup budget headroom confirmed per ECU class
- [ ] Algorithm decided per ECU class: ECDSA, ML-DSA or dual

KEYS AND OTP

- [ ] Public key anchor location and write protection defined
- [ ] Root and code-signing key hierarchy defined, including per-family scope
- [ ] Revocation mechanism (key slots or counter) defined in the OTP layout
- [ ] Dev key disablement at end of line defined
- [ ] Anti-rollback counter checked at flashing

PROCESS

- [ ] Backend signing HSM, access control and audit logging in place
- [ ] TARA updated for the new boot chain

0 of 10 done

NEXT

Open questions and what each one decides.

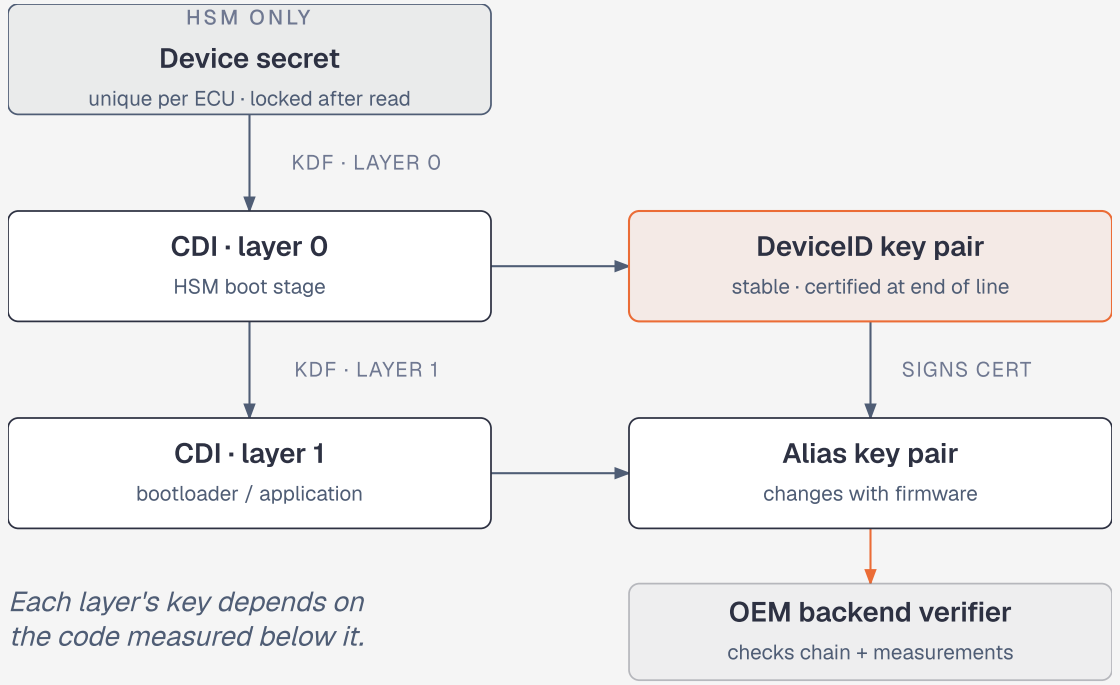
OPEN QUESTION	WHAT THE ANSWER DECIDES
AES-CMAC throughput on TC4xx for 2 MB?	The delta, and with it every per-class decision.
Startup time budget per ECU class?	Whether the delta fits, and ECDSA vs ML-DSA vs dual.
Any TC4xx ECU class with images much larger than 2 MB?	Whether the delta grows with size, and where staged verification is needed.
Which ECUs remain on TC3xx or SHE-class HSMs?	The size of the remaining hybrid fleet and its gate controls.

Next step: run the CMAC benchmark, collect budgets per ECU class, then apply the decision rule.

Signing keys are per family. Per-ECU keys come from DICE.

	SECURE BOOT	DICE
Question	May this code run?	What exactly runs on this ECU?
Checked by	The ECU itself, enforced	A remote party: backend, gateway
Keys	One code-signing key per family, in the backend	Device secret and key pairs per ECU, created on the ECU

Secure boot stops bad code from running. DICE lets the ECU prove what it booted. They fit together.



Adding DICE to the TC4xx boot chain.

1. **HSM is the DICE root.** Its boot stage reads the per-ECU device secret from OTP or a protected slot, derives the first CDI, then locks the secret until reset.
2. **Verify first, then measure.** Reuse the hash from signature verification as the measurement. DICE adds almost no hashing time.
3. **Measure signer and version for derivation.** The identity stays stable across signed updates. The exact image hash still goes into the alias certificate.
4. **Defer alias key generation.** Attestation is not needed before the first CAN message, so the startup budget is untouched.
5. **Certify DeviceID at end of line** with a separate device-identity CA, outside the code-signing tree.

WHAT IT ENABLES

- Attestation before OTA or diagnostic unlock
- SecOC keys and TLS certs issued only to healthy ECUs
- Clone detection through a certified identity
- Hardware-bound UDS 0x29 authentication

PLAN CHANGES

- OTP: device-secret slot with lock after read
- Backend: device-identity CA with per-device revocation
- End of line: DeviceID export and certificate issue
- TARA: attestation as its own claim

To confirm with Infineon: PUF availability as the device secret source, and lock-after-read support in the HSM firmware. Call the DICE secret “device secret” in documents to avoid a clash with UDS diagnostics.